REGRESSION

Oversized admonitions

A regression fixture for the `breakable: false` policy on admonitions. The warning below is deliberately long enough that on most page sizes it cannot fit between the surrounding prose blocks. With `breakable: false`, Typst will push the block onto its own page rather than splitting the label from the body — which is the trade-off this fixture verifies stays visually acceptable.

Open the rendered PDF and check: the warning block is intact (label above body, no mid-block page boundary) even if it has to start on a new page.

Some prose before the warning, so the layout has to make a real decision about where to place the block. Two short paragraphs of body copy on the neutral paper — set in Archivo at 10.5pt with the standard 0.62em leading. Nothing surprising in the type.

Some more prose. Long enough to push the start of the warning meaningfully into the middle of the page rather than at the top, so Typst's break choice is interesting. The warning has to either start in place and break across pages (the old behaviour), or push to a new page (the new behaviour).

WARNING

Sizing the pool is harder than it looks. Thread pools that hand out workers faster than the downstream API can absorb them are just a denial-of-service generator pointed at your own infrastructure. Add a bounded queue and a backpressure signal before you raise `max_workers`.

Common rules of thumb that almost always need adjustment in practice:

- For pure CPU-bound work on the CPython interpreter, the GIL means threads do not give you parallelism; use processes. The exception is C-extension code that releases the GIL (NumPy, image decoders, the hashlib primitives, most database drivers) — for those, threads do scale up to a point.
- For I/O-bound work, the classic `min(32, os.cpu_count() + 4)` heuristic comes from the CPython standard library and is reasonable but not load-aware. Measure the latency distribution of your downstream calls before you commit to a number.
- For mixed workloads, separate the pools. A single pool sized for the slowest dependency will under-utilise CPU on the rest.

The Python docs (PEP 703, the `concurrent.futures` module guide, and the `threading` chapter of the language reference) all hedge on specific values for good reason — the right number depends on your hardware, your downstream services, and the shape of your traffic. Treat any concrete number you find on the internet as a hypothesis to measure, not an answer to copy.

Things that look like they should help but usually don't:

- Adding more threads when the bottleneck is downstream latency. The extra threads just queue up; throughput does not change and tail latency gets worse.
- Reducing the pool size when CPU usage is "too high". A pool that hits 100% CPU during steady-state work is usually a sign that the worker count is well-matched to the work; the problem (if any) is upstream in the request shape, not the pool.
- Using `ProcessPoolExecutor` for I/O-bound work. The process spawn cost dominates anything you save on the GIL, which wasn't your bottleneck to begin with.

If the warning has reached this paragraph and is still in one piece on the page (or has started cleanly on a new page), `breakable: false` is doing its job.

After the warning, more prose to confirm the layout flows correctly back into the body column. This paragraph should appear immediately below the warning block, in the same column, with normal body spacing above.